RL-Course 2025/26: Final Project Report

Nils Klute* Frieder Wizgall† Georg Tirpitz‡

February 26, 2026

1 Introduction

The success of reinforcement learning can largely be attributed to its development and validation in game-like environments—ranging from early achievements on Atari games [8] to landmark results in complex strategy games such as Go [11]. Today, reinforcement learning remains a key approach for learning complex behaviors, and it has also become an important component in modern machine learning pipelines, including the fine-tuning of large language models to better align with user needs [10].
In this project, we focus on the Pendulum Upswing Environment [12] and the Hockey environment provided by the Autonomous Learning Group [7]. Both are based on Gym/Gymnasium, a standard API for reinforcement learning. While the pendulum problem provides an environment for debugging and testing, the main focus of this work is the Hockey Environment. It is a two-player air-hockey setting, the objective is to defeat an opponent by scoring a goal. Unlike many traditional benchmark tasks that optimize the cumulative discounted rewards in a stationary environment, a two-player game introduces additional challenges: The optimal action depends on the opponents behavior and performance is strongly influenced by the opponents the agent encounters during training. As a result, effective training requires careful choices about opponent selection and evaluation protocols.

To tackle this environment, we implemented and compared multiple reinforcement learning algorithms spanning several generations of methods—from value-based and actor–critic baselines such as Rainbow [5] and DDPG [6] to improved variants such as Fast-TD3 [3], and finally to more recent, model-based approaches such as TD-MPC2 [4]. We trained agents both against built-in opponents and through batched self-play/group training. We then conducted a comprehensive evaluation by measuring average win rates across pairwise matchups. Finally, we discuss the strengths and weaknesses of the different approaches and summarize the main conclusions.

2 Methods

2.1 Rainbow DQN

DQN base algorithm. Our implementation of Rainbow DQN is based on an implementation of DQN from the Self Driving Cars lecture held by the Autonomous Vision Group. DQN [9] is based on Q-Learning, but uses function approximation to estimate the optimal Q-Function $Q_\theta(s, a) \approx Q^*(s, a)$ via a neural network. The action space needs to be discrete, as the network outputs a Q-Value for each

*Implemented Rainbow DQN
†Implemented TD3
‡Implemented TD-MPC2

action a. For our purpose, we use the conversion given by the Hockey environment. The policy is then given implicitly as $\pi_\theta(s) = \arg\max_a Q_\theta(s, a)$. It is trained using the temporal-difference (TD) Bellman target.
Two key ingredients make the network trainable: First, train on samples $(s_t, a_t, r_t, s_{t+1}, d_t) \sim U(D)$ drawn uniformly from a replay buffer D to break correlations between samples in a batch. Second, utilize a fixed target network with weights θ' that estimates the Bellman target. This reduces oscillations and stabilizes training.
Now the Q-Network is optimized by minimizing the following objective

$$\mathcal{L}(\theta) = \mathbb{E}_{(s_t,a_t,r_t,s_{t+1},d_t)\sim U(D)}\Big[\big(r_t + \gamma(1-d_t)\ \arg\max_{a'} Q_{\theta'}(s_{t+1}, a') - Q_\theta(s_t, a_t)\big)^2\Big] \tag{1}$$

where d_t is the terminal flag.
As a design choice, instead of hard target network updates, we use slowly updated target networks via Polyak averaging

$$\theta' \leftarrow \tau\theta + (1-\tau)\theta' \tag{2}$$

where $0 < \tau \ll 1$ controls the tradeoff between speed and stability of learning.
Regarding the neural network architecture, we chose a small Multi Layer Perceptron with two hidden layers, each consisting of 128 units.

Rainbow extensions. We implemented all 6 extensions to DQN proposed by Rainbow, namely Double Q-Learning (DQ), Dueling Networks (DN), Multi-Step Learning (N-STEP), Noisy Nets (NN), Prioritized Replay (PR) and Distributional RL (DRL). As experimentation with PR was not sucessfull, and DRL did not provide extra gains in terms of speed and stability, we will omit a formal description for the sake of clarity.
(1) Double Q-Learning: Standard Q-Learning can suffer from overestimation bias due to the maximization step in the target. Double-Q Learning alleviates this by separating action selection from evaluation. The policy network with parameters θ is used for action selection, while the target network with parameters θ' estimates the action value. The altered objective can be described as

$$\mathcal{L}(\theta) = \mathbb{E}\Big[\big(r_t + \gamma(1-d_t)\, Q_{\theta'}(s_{t+1}, \arg\max_{a'} Q_\theta(s_{t+1}, a')) - Q_\theta(s_t, a_t)\big)^2\Big]. \tag{3}$$

The decoupling mechanism reduces overestimation and can improve performance.
(2) Dueling Architecture: Roughly speaking, the value of a particular action in a particular state can be attributed to being either in a good state in general or executing a useful action. Sometimes it can be hard to estimate both jointly with a single Q-Value. The Dueling Architecture addresses this problem by introducing a common backbone followed by two separate downstream computation paths. The value stream uses the backbone features to compute the value of a specific state. The advantage stream computes the value of an action in a particular state relative to all other actions. Those streams are combined to compute the final Q-Value as follows:

$$Q_\theta(s, a) = v_\eta(f_\xi(s)) + a_\psi(f_\xi(s), a) - \frac{1}{N_{\text{actions}}}\sum_{a'} a_\psi(f_\xi(s), a') \tag{4}$$

Here, ξ, η, ψ are the parameters of the backbone, value and advantage streams, respectively. $\theta = \{\xi, \eta, \psi\}$ is their concatenation.
(3) Multi-Step Learning: Standard Q-Learning collects the current reward and bootstraps the target one

step into the future. It can be challenging and/or inefficient to backpropagate rewards towards earlier states, especially in environments where rewards are sparse. Multi-Step Learning tackles this by following the agents trajectory, incorporating discounted rewards n steps ahead into the target. The bootstrap is then computed at the n-th look ahead step. Let us define the truncated n-step return as

$$R_t^{(n)} := \sum_{k=0}^{n-1} \gamma^{(k)} R_{t+k+1}, \qquad \text{with } \gamma^{(k)} := \gamma^k \prod_{i=1}^{k} (1 - d_t). \tag{5}$$

Then the alternative multi-step loss can be formulated as

$$\mathcal{L}(\theta) = \mathbb{E}\Big[\big(R_t^{(n)} + \gamma^{(k)} \arg\max_{a'} Q_{\theta'}(s_{t+n}, a') - Q_\theta(s_t, a_t)\big)^2\Big]. \tag{6}$$

Using a suitable tuned n, learning efficiency can be improved.
(4) Noisy Nets: The standard DQN uses scheduled ϵ-greedy strategy for exploration. This approach has obvious limitations, as it is difficult to estimate the amount of exploration needed at a given time in advance. Noisy Nets transfers the exploration mechanism to the neural network. It uses a noisy linear layer, consisting of a deterministic and stochastic stream

$$\boldsymbol{y} = (\boldsymbol{b} + \mathbf{W}\boldsymbol{x}) + (\boldsymbol{b}_{noisy} \odot \epsilon^b + (\mathbf{W}_{noisy} \odot \epsilon^w)\boldsymbol{x}), \tag{7}$$

where ϵ^b and ϵ^w are random variables and $\odot$ represents the element-wise product. $\boldsymbol{b}_{noisy}$ and $\mathbf{W}_{noisy}$ are learnable parameters, thus the network can learn to ignore the exploration noise from the stochastic stream over time.

Hyperparameter Optimization Hyperparameter Optimization was conducted manually, training against the weak opponent. For the sake of clarity, we will not report Pooled Training Tuning (see Section 2.4). We initialized with standard Rainbow hyperparameters. Extensions were added step by step.
Comparing the base algorithm with the extensions, N-STEP was the only one able to stabilize, showing for a wide range of choices for $3 \leq n \leq 8$. Regarding DN, a reduction of the target network update speed from $\tau = 0.005 \rightarrow 0.0005$ was necessary. Turning on DQ did not destabilize training, but slightly degraded performance. Experiments with target network update speed $\tau = 0.0005 \rightarrow \tau = 0.001$, batch-size $B = 32 \rightarrow B = 64$ and Adam's momentum $\beta_1^{Adam} = 0.9 \rightarrow \beta_1^{Adam} = 0.5$ seemed sufficient to restore performance. As the last step, integrating NN did not require further adjustments, we used the default initialization $\sigma_0 = 0.5$ from the paper.

2.2 Fast TD3

DDPG base algorithm. Our implementation of Fast-TD3 is built on the lecture implementation of the Deep Deterministic Policy Gradient (DDPG) algorithm. DDPG uses a critic $Q_\theta(s, a)$ trained with a temporal-difference (TD) Bellman target. For a transition $(s_t, a_t, r_t, s_{t+1}, d_t)$ from the replay buffer (with terminal flag d_t), the target is

$$y_t = r_t + \gamma(1 - d_t)\, Q_{\theta'}\big(s_{t+1}, \pi_{\phi'}(s_{t+1})\big), \tag{8}$$

and the critic is optimized by minimizing

$$\mathcal{L}_{\text{critic}}(\theta) = \mathbb{E}\Big[\big(Q_\theta(s_t, a_t) - y_t\big)^2\Big]. \tag{9}$$

Since actions are continuous, $\arg\max_a Q(s,a)$ cannot be computed by enumeration. Instead, DDPG trains an actor $a = \pi_\phi(s)$ to maximize the critic:

$$\mathcal{L}_{\text{actor}}(\phi) = -\mathbb{E}\big[Q_\theta(s, \pi_\phi(s))\big]. \tag{10}$$

To stabilize bootstrapping, DDPG uses slowly updated target networks (θ', ϕ') via Polyak averaging:

$$\theta' \leftarrow \tau\theta + (1-\tau)\theta', \qquad \phi' \leftarrow \tau\phi + (1-\tau)\phi', \qquad 0 < \tau \ll 1. \tag{11}$$

TD3 upgrades. Classic DDPG can be unstable due to critic overestimation and the actor exploiting critic errors. Twin-Delayed DDPG (TD3) alleviates these issues with three key upgrades.
(1) Clipped double Q: TD3 trains two critics and uses the minimum for the target to reduce overestimation:

$$y_t^{\text{TD3}} = r_t + \gamma(1-d_t) \min_{i\in\{1,2\}} Q_{\theta_i'}\big(s_{t+1}, \tilde{a}_{t+1}\big). \tag{12}$$

(2) Target policy smoothing: the target action is perturbed with small clipped Gaussian noise,

$$\tilde{a}_{t+1} = \pi_{\phi'}(s_{t+1}) + \epsilon, \qquad \epsilon \sim \text{clip}\big(\mathcal{N}(0,\sigma^2), -c, c\big), \tag{13}$$

which reduces sensitivity to spurious sharp peaks in the learned Q function.
(3) Delayed policy updates: the actor (and typically target networks) are updated only every P critic updates, which prevents the actor from chasing transient critic errors and stabilizes learning.

Fast-TD3 upgrades used in this project. TD3 is more stable than DDPG, but it is also more computationally expensive due to the second critic. Fast-TD3 improves wall-clock performance and robustness. We implement two upgrades from Fast-TD3:
(1) Parallel environments: we collect experience from many environments in parallel, which fills the replay buffer quickly with diverse, recent transitions and enables large-batch training for efficient GPU utilization and more stable gradients. To do this we perform batched inference and observations while the update stays the same.
(2) Distributional critic: instead of predicting only the expected return $Q(s,a)$, we model a return distribution $Z(s,a)$, which can provide a richer training signal and improve stability. Additionally we crop the quantiles be deleting the highest 10 quantiles to further limit the overestimation bias.

Hyperparameter Optimization For hyperparameter optimization, we used the Bayesian sweep functionality of Weights & Biases (W&B) [2] and ran the sweep in a distributed manner across multiple server nodes. The Bayesian surrogate model enabled the search to focus quickly on promising regions of the hyperparameter space, leading to fast convergence to a stable configuration. At the same time, distributing trials over many nodes maximized parallel throughput, while W&B centrally coordinated experiment scheduling and updated the Bayesian model based on results from all workers.

2.3 TD-MPC2

TD-MPC2 base algorithm. TD-MPC2 is a model-based reinforcement learning method that combines temporal-difference learning with Model Predictive Control (MPC) [4]. At each decision step, the agent is rolling out a finite action sequence $(a_t, \ldots, a_{t+H-1})$ and selects actions that maximize predicted return over a horizon H under a learned world model. In practice, this planning procedure is performed by sampling and optimizing candidate action sequences.

A key idea of TD-MPC2 is to learn an implicit rather then explicit world model. Traditional approaches like Dreamer reconstruct future observations in observation/state space in an encoder-decoder scheme. In contrast, Td-MPC2 removes the decoder and performs planning directly in a latent space z. From latent states, the model directly predicts next latent states, rewards, values, and a policy prior. This avoids explicit decoding of future observations, reduces planning cost, and focuses model capacity on control-relevant features rather than realistic predictions in the observation space. We denote the latent forward dynamics model as lf.

Networks. Following TD-MPC2, we model planning and value estimation with five learned components that operate in latent space. Let s denote the observation (state), a the action, z the latent representation. We have 5) distinct networks, namely the i) encoder $z = h(s)$ that maps observations to latent representations ii) The model $z' = \mathrm{lf}(z, a)$ for the forward latent dynamics iii) the reward model $\hat{r} = R(z, a)$ that predicts the transition reward iv) the q-model $\hat{q} = Q(z, a)$, and v) the policy prior $\hat{a} = p(z)$ that predicts the actions for planning. In our implementation all five components are parameterized as 2-layer MLPs and hidden layer width 512, using a latent dimension of $z \in \mathbb{R}^{256}$. The critic uses a double-Q architecture (for upgrades later ensembles of 2 or more), and the policy prior outputs normalized actions in $[-1, 1]^{d_a}$ with a final $\tanh$.

Objectives. TD-MPC2 optimizes two distinct objectives: a *model objective* for the latent world model and value prediction, and a *policy objective* for the policy prior used during planning. In the following, we omit the task embedding e for readability.

Model objective. The encoder h, latent forward dynamics lf, reward model R, and value model Q are trained jointly using a temporally weighted multi-step objective:

$$\mathcal{L}_{\text{model}}(\theta) = \mathbb{E}_{(s,a,r,s')_{0:H}\sim\mathcal{B}}\left[\sum_{t=0}^{H}\lambda^t\Big(\|z'_t - \mathrm{sg}(h(s'_t))\|_2^2 + \mathrm{CE}(\hat{r}_t, r_t) + \mathrm{CE}(\hat{q}_t, q_t)\Big)\right], \tag{14}$$

where $\theta = \{h, \mathrm{lf}, R, Q\}$, $\mathrm{sg}(\cdot)$ denotes stop-gradient, and $(z'_t, \hat{r}_t, \hat{q}_t)$ are the latent next-state, reward, and value predictions produced by the model. The first term enforces latent consistency, while the second and third terms train reward and value prediction (implemented as discrete regression with cross-entropy in TD-MPC2).

Policy objective. The policy prior p is trained to maximize predicted value while maintaining exploration through an entropy term:

$$\mathcal{L}_{\text{policy}}(\phi) = -\mathbb{E}_{(s,a)_{0:H}\sim\mathcal{B}}\left[\sum_{t=0}^{H}\lambda^t\left(\alpha\, Q(z_t, p(z_t)) - \beta\,\mathcal{H}(p(\cdot \mid z_t))\right)\right], \tag{15}$$

with latent rollout

$$z_{t+1} = \mathrm{lf}(z_t, a_t), \qquad z_0 = h(s_0). \tag{16}$$

Here, α and β are the hyperparameters for value maximization and entropy regularization. Gradients of $\mathcal{L}_{\text{policy}}$ are taken only with respect to the policy prior parameters ϕ.

Planning objective (MPC/MPPI). In addition to the model and policy training objectives, TD-MPC2 uses an online planning objective at inference time. At state s_t, MPPI samples action sequences from a time-dependent Gaussian and optimizes its parameters (μ, σ) to maximize predicted return under latent rollouts:

$$\mu^*, \sigma^* = \arg\max_{\mu,\sigma}\ \mathbb{E}_{(a_t,\ldots,a_{t+H-1})\sim\mathcal{N}(\mu,\sigma^2)}\left[\sum_{h=0}^{H-1}\gamma^h\hat{r}_h + \gamma^H Q(z_H, p(z_H))\right], \tag{17}$$

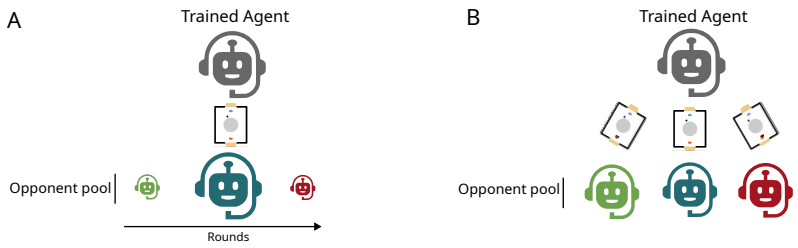


Figure 1: Sketch of the pooled training logic. In A we see the sequential, and in B the parallel pooled training logic.

where $z_0 = h(s_t)$, $z_{h+1} = \mathrm{lf}(z_h, a_{t+h})$, and $\hat{r}_h = R(z_h, a_{t+h})$. The first action of the optimized sequence is executed, and the procedure is repeated at the next time step (receding-horizon control). The Q network is optimized in the same way as stated above for the DDPG base algorithm.

TD-MPC2 upgrades. Further attempts to improve the learning of TD-MPC2, we tried out two changes. *(1)* Changing to Muon optimizer [1]. There is growing evidence that Muon leads to more stable and faster convergence whilst adding minimal computational overhead. *(2) Epistemic uncertainty for exploration:* We add an epistemic exploration bonus to the MPC planning score. For each sampled action sequence, the planner evaluates

$$J_{\text{plan}} = \sum_{h=0}^{H-1} \gamma^h \hat{r}_h + \gamma^H Q(z_H, p(z_H)) + \beta_{\text{epi}}\, U_{\text{epi}}, \tag{18}$$

where U_{epi} is a trajectory-level uncertainty estimate and β_{epi} is linearly scheduled over training. In our implementation, U_{epi} can be computed from (i) critic disagreement (e.g. $|Q_1 - Q_2|$), (ii) variance across Q-ensemble heads, and/or (iii) disagreement between online and target-model latent/reward rollouts. The resulting uncertainty term is optionally normalized across sampled trajectories and added to the MPPI score, biasing planning toward informative (uncertain) trajectories.

Hyperparameter optimization. For tuning, we used the same W&B [2] sweeps described in TD3. We have split the hyperparameter tuning in two stages: We first tuned the base version without upgrades on the weak opponent. As a second step, we fine-tune the MPC related hyperparameters on the strong opponent, namely horizon length H and τ, while fixing the remaining hyperparameters from the first stage. In addition we tune the new hyperparameters introduced by the upgrades.

2.4 Sequential and Parallel Pooled Training

To elevate our agents skill and robustness it is not sufficient to simply train against the strong and weak basic opponents provided by the environment. Therefore we propose a method for training an agent π against a pool of agents. Let us denote the opponent pool as $\Pi' = \{\pi'_1, ..., \pi'_K\}$, consisting of the basic opponents and a set of custom trained agents using our algorithms in the standard setting against a single opponent. For training, we initialize with an agent pretrained against the strong basic opponent in single opponent training. In our setting, a full training run consist of N rounds. For each round $i \in \{1, .., N\}$, we randomly draw the current opponent π'_l with $l \sim Cat(\boldsymbol{p})$ from the pool. Here $\boldsymbol{p} = \{p_1, ..., p_K\}$ denotes the probabilities of choosing the respective opponent. We then train π for M episodes against opponent π'_l. After the current training iteration has completed, we evaluate our model against the pool. Let $w_l^{(i)}$ denote the average winrate of agent π against opponent π'_l in training round

i. We then compute the average winrate against the pool as $w^{(i)} = \frac{1}{K}\sum_{l=1}^{k} w_l^{(i)}$. We save the current agent only if $w^{(i)} > w^{(i-1)}$, where $w^{(0)} = 0$. Given a diverse opponent pool, this should lead to an robust agent that can defend against a wide range of strategies.
A disadvantage of this approach is a potentially circular pattern of training. The agent develops a good strategy against some opponent, but suffers catastrophic forgetting as soon as it gets switched out. Thus we extend the approach to parallel environments, where we train an agent in a set $\mathcal{E} = \{\mathcal{M}_1, ..., \mathcal{M}_P\}$ of environments in parallel. At every train iteration i, we select an opponent π'_{l_p} with $l_p \sim Cat(\boldsymbol{p})$ for each environment $\mathcal{M}_p$ in the manner described above. Then we simply train π in all environments $\mathcal{E}$ against $\{\pi'_{l_1}, ..., \pi'_{l_P}\}$ simultaneously for M rounds, evaluate against the pool and save as usual.

3 Results

First we report and compare the algorithms' performance in the pendulum swingup environment and the Hockey environment.
The pendulum environment was primarily used for testing the functionality and debugging Rainbow extensions. Here we report moving average rewards as defined below. The hockey environment was the main focus of this work. We show moving average rewards and final winrates against the weak and strong opponents of agents that were only trained against the weak and strong opponents for comparability between the algorithms. Later greatly improved our algorithms using the sequential and parallel pooled training method which resulted in the agents we submitted to the tournament.
All reported moving average rewards are computed over the history of the last $k = \frac{M}{100}$ episodes with M number of episodes as $\bar{r}_t = \frac{1}{k}\sum_{i=t-k+1}^{t} R_i$, where R_i denotes the rewards in episode i. Final winrates w against the basic opponents are computed as an average over 5000 episodes.

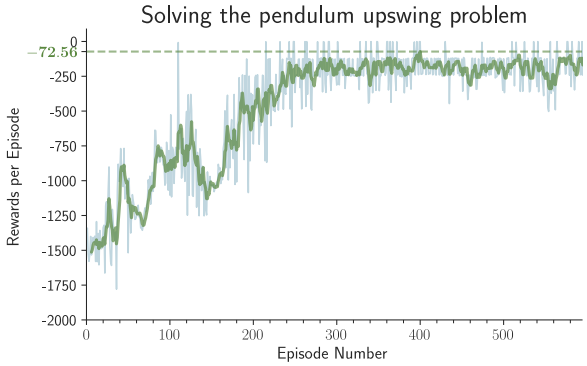


Figure 2: Rewards per episode plot of Rainbow with all extensions for the pendulum environment. We achieve a moving average rewards per episode $\bar{r}_t = -72.56$ within 400 episodes.

3.1 Rainbow

The base algorithm and all extensions in isolation are able to solve the pendulum environment consistently. We report results in Figure 2 with all four extensions combined. We achieve a moving average rewards per episode $\bar{r}_t = -72.56$. For the hockey environment, we show results in Figure 3. The algorithm beats the weak opponent consistently with a moving average reward $\bar{r} = 9.29$ and a final winrate $w = 0.97$. Against the strong opponent, we obtain a moving average reward $\bar{r} = 4.20$ and a final winrate $w = 0.69$.

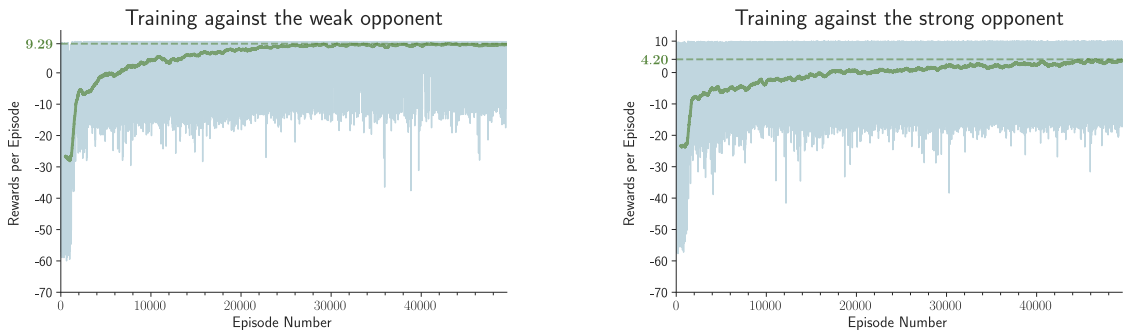


Figure 3: **Left**: Rewards per episode plot of Rainbow with all extensions against the weak opponent. We obtain a moving average reward $\bar{r} = 9.29$ and a winrate $w = 0.97$ within 50000 episodes. **Right**: Rewards per episode plot of Rainbow with all extensions against the strong opponent. We obtain a moving average reward $\bar{r} = 4.2$ and a winrate $w = 0.69$ within 50000 episodes.

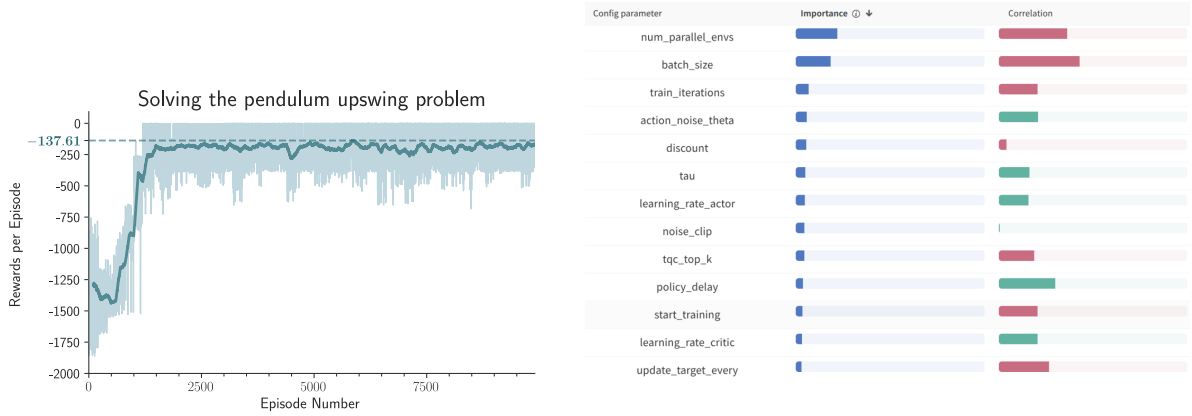


Figure 4: **Left:** Reward per episode plot of Fast-TD3 trained on the pendulum task. We achieve a moving average reward $\bar{r} = -137.61$ within 1500 episodes. **Right:** The hyperparameter importances as determined by the Wandb tuning library. We can see that the number of parallel environment the batch size and the training iterations are most important.

3.2 TD-3

Training the TD-3 agent using the hyperparameters from the wandb sweep in the pendulum and hockey environment resulted in good performance. The top moving average result in the pendulum environment is $\bar{r} = -137.61$ within 1500 episodes. In the hockey environment TD-3 achieves a moving average reward of 9.17 (with a surprisingly low winrate $w = 0.36$, which could be greatly improved up to 0.99 using the parallel pooled training method), which is similar to the performance achieved against the strong opponent ($\bar{r} = 9.17$, $w = 0.63$, which could also be improved up to 0.99 by parallel pooled training). To note is also the speed in which these results can be obtained as both runs of 5000 episodes ran in under 2 hours of wall clock time on the TSMC cluster. The detailed results can be seen in figure 5.

3.3 TD-MPC2

We developed the base algorithm of TD-MPC using the Pendulum environment with minimal tuning effort. Here we obtain a moving average reward $\bar{r} = -56.66$ within 100 episodes. Next, we moved

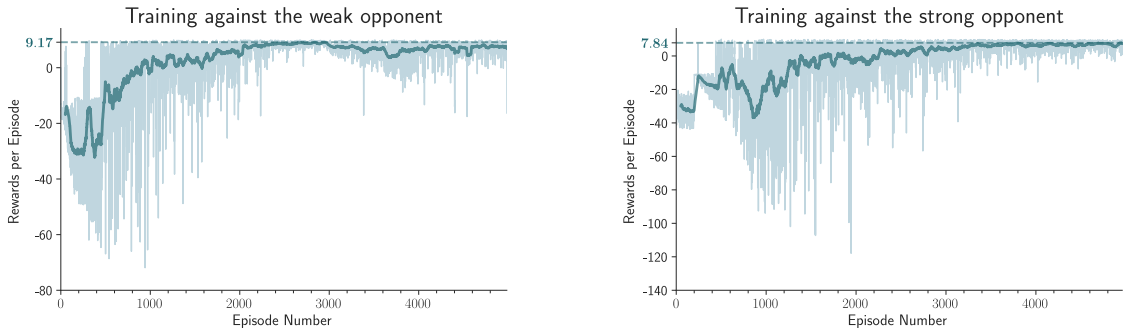


Figure 5: Fast-TD3 against the weak hockey opponent. We achieve a moving average reward $\bar{r} = 9.17$ and a winrate $w = 0.36$ within 5000 episodes. **Right**: Reward per episode plot of Fast-TD3 trained against the strong opponent. We achieve a moving average reward $\bar{r} = 7.84$ and a winrate $w = 0.63$ within 5000 episodes.

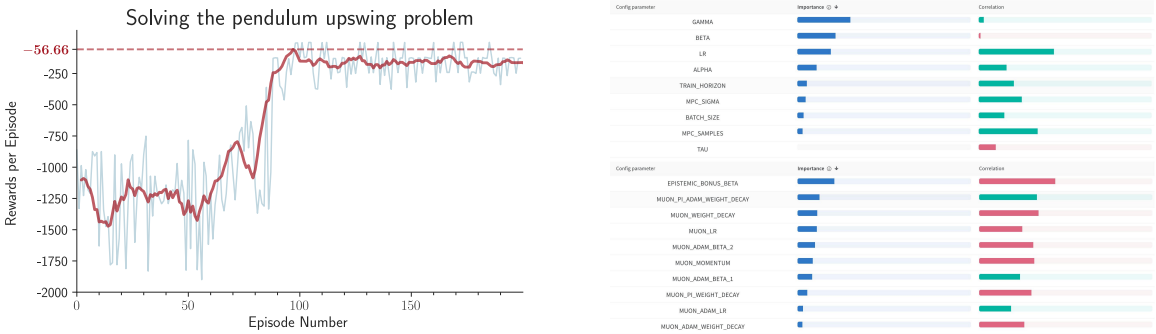


Figure 6: **Left**: Reward per episode plot of TDMPC2 trained on the pendulum task. We achieve a moving average reward $\bar{r} = 56.66$ within 100 episodes. **Right**: On top are the parameter importances from the first tuning stage, and below are the importances for the second tuning stage.

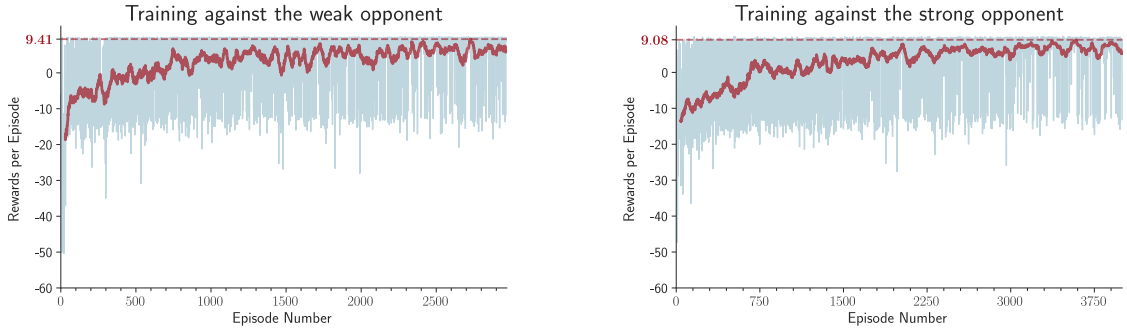


Figure 7: TD-MPC2 against the weak hockey opponent. We achieve a moving average reward $\bar{r} = 9.41$ and a winrate $w = 0.8525$ within 3000 episodes. **Right**: Reward per episode plot of TD-MPC2 trained with Muon and epistemic-based exploration in the Hockey environment against the strong opponent. We achieve a moving average reward $\bar{r} = 9.08$ and a winrate $w = 0.9035$ within 4000 episodes.

to the hockey environment, tuning the hyperparameters of the first stage on the weak opponent. The importance of parameters shown in the plot provided by W&B indicate that γ, β, learning rate η, α and train horizon H seem to have the highest impact on the moving average reward $\bar{r}$. The performance

against the weak opponent is shown in Figure 7 on the left. We report moving average reward $\bar{r} = 9.41$ saturating near maximum possible reward 10 within 3000 episodes and a winrate of $w = 85.25\%$ against the weak opponent. Proceeding to the second stage, we tuned hyperparameters on the strong opponent. We see that the most important hyperparameters are the epistemic exploration coefficient β_{epi}, followed by the Muon related hyperparameters. The performance of the upgraded TD-MPC2 is shown in Figure 7 on the right. We see a moving average reward $\bar{r} = 9.08$ saturating around 3000 episodes with a winrate of $w = 90.35\%$ against the strong opponent. We compare this to the base algorithm saturating at $\bar{r} = 8.02$ within 3000 episodes.

3.4 Opponent Pool

During opponent pool training, we accumulated a pool of 20 agents with promising results. We iterativly trained against them to improve our agents. For the sake of clarity, we only report a subset of agents showing the development of skills over time. Average winrates are shown in Table 1. We can see that Fast TD-3 performs best in the pool setup and is able to beat pool agents consistently.

Pool Performance		
Algorithm	Version	$w^{(i)}$
Rainbow	1	0.19
Rainbow	2	0.09
Rainbow	3	0.64
Fast TD-3	1	0.55
Fast TD-3	2	0.71
Fast TD-3	3	**0.91**
TDMPC2	1	0.46
TDMPC2	2	0.54
TDMPC2	3	0.45

Table 1: Here we report average winrate $w^{(i)}$ of selected agents against our pool of 20 collected agents during pool training. Versions 1 & 2 are agents selected to illustrate the evolution of the algorithm's performance during pool training. Version 3 is the final version we submitted to the competition.

4 Discussion

In this report, we implemented three different Deep RL algorithms, namely Rainbow, Fast-TD3 and TD-MPC2. We measured their performance in the Pendulum environment as maximum rewards per episode and report winrates against the opponent for the Hockey environment. We will omit a discussion for Pendulum, as it only served as implementation and debugging aid. Focusing on the hockey environment, we will first analyze performance in the training setup against a single opponent for each algorithm, then proceed to the pooled training results.

Rainbow. We can see that the rainbow algorithm consistently outperforms the weak agent, but struggles to achieve top performance on the strong opponent. This could be attributed to several factors. First, we tuned hyperparameters only on the weak opponent. Further tuning could have a significant impact on

model performance. Second, we used a neural network with 2 Hidden Layers and 128 units each. We did not experiment with varying network sizes. We identify this as a strong limitation, as bigger networks should have the necessary capacity to allow for more sophisticated strategies.
Regarding the pool performance, Rainbow was able to develop strategies to defend against a pool of agents, including past versions that were trained on the weak agent or other pools. Performance eventually hit a ceiling compared to Fast-TD3. Here we identify three limitations of our approach. First we did no extensive tuning on the pool and further emphazise the small network choice. A bigger network could have the capacity to defend against opponents with varying strategies. A third issue is the chosen discretization. Qualitiative evaluation shows that the Rainbow agent lacks the agility seen for Fast-TD3. A finer discretization could resolve this in parts.

Fast TD3. The TD3 agent appears to strike a good balance between complexity and efficiency for the hockey environment. Agents trained in 1v1 play against the built-in baseline opponents quickly learned to gain good reward. Moreover, refining these agents with the iterative pooled training method reliably improved performance, enabling them to outperform the other agent types while keeping the computational load low by parallelizing across many environments. One limitation, however, was a lack of exploration of more "creative" strategies, such as defending from within the goal or using the wall to set up shots. Instead, the policy largely optimized for fast, accurate shooting, which may have capped its performance against more sophisticated tournament opponents. This behavior could stem from the exploration scheme (e.g., mostly random action noise) and/or from the limited network capacity of both the actor and critic.

TD-MPC2. We see that the non-upgraded version of TD-MPC2 shows a strong performance against the weak opponent, and the upgraded TD-MPC2 shows a similar performance against the strong opponent. However, it seems that the average reward seems less stable as for the other algorithms. Nevertheless, we can view the environment opponents as solved by TD-MPC2. We see that the performance drops then in the opponent pool. One major issue is that TD-MPC2 is on the upper end of complexity and size. The inference time, which played a huge factor in our opponent pool setup, was much slower for TD-MPC2 than for the other methods, limiting the time that could be trained on the full opponent pool. Also TD-MPC2 did not profit from the parallel environments opponent set up, as this seemed to interfere with the rollouts, slowing down the training time even further. TD-MPC2 excelled against a fixed opponent but did not seem to generalize well to a bigger set of opponents. In the TD-MPC2 paper [4] they highlighted very strong generalization, but they had parameter sizes up $317M$ parameters. To yield better generalization over multiple distinct opponents, we suppose that with a larger latent space model and a third stage of tuning, we would left the catastrophic forgetting stage earlier would have seen stronger generalization over multiple opponents. However, this would have demanded even more time and resources.

In summary, we successfully implemented and compared three different algorithms using the pendulum and the hockey environment. Our algorithms ranged from lightweight Rainbow implementations to heavyweight TDMPC2 models. We trained on the weak and strong opponents successfully and proposed a method for Pooled Training. We saw that Fast TD-3 showed superior performance in our Pool setting. Finally in the course-internal tournament we could reach rank 31 with the hockey007-ddpg (TD3) agent.

5 Usage of AI

5.1 Report

AI tools were used only for grammar and spelling checks. All report content was originally written by members of the team.

5.2 Code

The overall structure, logic, and design of the codebase were developed by the team. AI was used in a limited capacity to help generate small utility functions and assist with debugging by prompting Codex 5.3 within the Visual Studio Code environment. Any AI-generated code was manually reviewed and verified by the team before being incorporated.

References

[1] E. AI, :, I. Shah, A. M. Polloreno, K. Stratos, P. Monk, A. Chaluvaraju, A. Hojel, A. Ma, A. Thomas, A. Tanwer, D. J. Shah, K. Nguyen, K. Smith, M. Callahan, M. Pust, M. Parmar, P. Rushton, P. Mazarakis, R. Kapila, S. Srivastava, S. Singla, T. Romanski, Y. Vanjani, and A. Vaswani. Practical efficiency of muon for pretraining, 2025.

[2] L. Biewald. Experiment tracking with weights and biases. `https://www.wandb.ai/`, 2020. Software available from wandb.ai.

[3] S. Fujimoto, H. Hoof, and D. Meger. Addressing function approximation error in actor-critic methods. In *International conference on machine learning*, pages 1587–1596. PMLR, 2018.

[4] N. Hansen, H. Su, and X. Wang. Td-mpc2: Scalable, robust world models for continuous control, 2024.

[5] M. Hessel, J. Modayil, H. V. Hasselt, T. Schaul, G. Ostrovski, W. Dabney, D. Horgan, B. Piot, M. G. Azar, and D. Silver. Rainbow: Combining improvements in deep reinforcement learning. In *AAAI*, 2018.

[6] T. P. Lillicrap, J. J. Hunt, A. Pritzel, N. Heess, T. Erez, Y. Tassa, D. Silver, and D. Wierstra. Continuous control with deep reinforcement learning. *arXiv preprint arXiv:1509.02971*, 2015.

[7] G. Martius. Hockey-env: A simple laser-hockey gym environment for rl agents. `https://github.com/martius-lab/hockey-env`, 2021. Accessed: 2024-05-20.

[8] V. Mnih, K. Kavukcuoglu, D. Silver, A. Graves, I. Antonoglou, D. Wierstra, and M. Riedmiller. Playing atari with deep reinforcement learning. *arXiv preprint arXiv:1312.5602*, 2013.

[9] V. Mnih, K. Kavukcuoglu, D. Silver, A. A. Rusu, J. Veness, M. G. Bellemare, A. Graves, M. Riedmiller, A. K. Fidjeland, G. Ostrovski, S. Petersen, C. Beattie, A. Sadik, I. Antonoglou, H. King, D. Kumaran, D. Wierstra, S. Legg, and D. Hassabis. Human-level control through deep reinforcement learning. *Nature*, 518(7540):529–533, Feb. 2015.

[10] L. Ouyang, J. Wu, X. Jiang, D. Almeida, C. Wainwright, P. Mishkin, C. Zhang, S. Agarwal, K. Slama, A. Ray, et al. Training language models to follow instructions with human feedback. *Advances in neural information processing systems*, 35:27730–27744, 2022.

[11] D. Silver, J. Schrittwieser, K. Simonyan, I. Antonoglou, A. Huang, A. Guez, T. Hubert, L. Baker, M. Lai, A. Bolton, et al. Mastering the game of go without human knowledge. *nature*, 550(7676):354–359, 2017.

[12] M. Towers, A. Kwiatkowski, J. Terry, J. U. Balis, G. D. Cola, T. Deleu, M. Goulão, A. Kallinteris, M. Krimmel, A. KG, R. Perez-Vicente, A. Pierré, S. Schulhoff, J. J. Tai, H. Tan, and O. G. Younis. Gymnasium: A standard interface for reinforcement learning environments, 2025.